Universidad CENFOTEC
Escuela de Fundamentos
FUN-01 Introducción a las Tecnologías de Información

Proyecto Integrador NanyBoard: Peluche Inteligente de Monitoreo y Seguridad Infantil
Avance 2: Prototipado y Sala de Experimentos

Andres Benavides
Fabricio Agüero
Daniel Festa

Docente: Yusselin Murcia Céspedes
Sección FCP0, Periodo C3-2026
13 de noviembre de 2026

Tabla de contenido

Introducción 3
Primera versión funcional del código 3
Herramientas y librerías 3
Estructura del programa 5
Lógica principal: máquina de estados con prioridades 5
Fragmento clave comentado 6
Cambios respecto al pseudocódigo del Avance 1 6
Montaje preliminar 7
Componentes utilizados 7
Tabla de conexiones (pinout preliminar) 7
Etapas del montaje 8
Sala de experimentos: pruebas de sensores y salidas 8
Procedimiento 8
Calibración de umbrales 10
Problemas encontrados y soluciones 11
Bitácora DiálogoX™ / IA: consultas técnicas, depuración y optimización 11
Limitaciones detectadas en la ayuda de la IA 12
Próximos pasos hacia la entrega final 13
Referencias 13
Anexo A. Código fuente del firmware 14
Anexo B. Scripts de la sala de experimentos 20

Introducción

En el Avance 1 se definió NanyBoard, un peluche que integra la placa IdeaBoard (ESP32) y el módulo IdeaSense Explorer para vigilar la habitación de un niño pequeño y avisar al cuidador ante humo o gas, llanto prolongado, caídas o temperaturas fuera del rango de confort. Este segundo avance corresponde a la fase de prototipado y sala de experimentos: se presenta la primera versión funcional del código en CircuitPython, el montaje preliminar de la IdeaBoard con la IdeaSense y los sensores externos, el plan y registro de pruebas de lectura de sensores y control de salidas, y la bitácora de interacción con inteligencia artificial utilizada para resolver dudas técnicas, anticipar errores y optimizar la lógica.

El documento también explica qué cambió respecto al pseudocódigo del Avance 1 y por qué, ya que varias decisiones de diseño se ajustaron al confrontar la idea original con las limitaciones reales del hardware.

Primera versión funcional del código

Herramientas y librerías

El firmware se programó en CircuitPython (versión 10.x, la estable para la IdeaBoard) y se carga mediante IdeaCode o Thonny. Se utilizan las librerías oficiales del ecosistema IdeaBoard y una librería de Adafruit para la comunicación inalámbrica:

- **ideaboard**: clase IdeaBoard, controla el LED RGB y los pines de la placa.
- **ideasense**: clase IdeaSense, da acceso a la matriz LED 5x5, a los botones A, B y C, al sensor de temperatura y humedad (SHT30), al sensor de luz (LTR-303) y al acelerómetro/giroscopio (LSM6DS3TR-C), todos por I2C.
- **analogio** y **pwmio** (incluidas en CircuitPython): lectura analógica del sensor de gas MQ-2 y del micrófono, y generación de tonos en el buzzer.
- **adafruit_minimqtt**: envío de lecturas y alertas por Wi-Fi mediante el protocolo MQTT hacia la aplicación del cuidador.

analogio: leer sensores analógicos

Un sensor **analógico** no responde “sí o no”. Entrega un voltaje que sube y baja según lo que mide. El MQ-2 da más voltaje cuanto más gas hay, y el micrófono da un voltaje que vibra con el sonido.

`analogio` convierte ese voltaje en un número de **0 a 65535**: 0 es 0 voltios y 65535 es 3,3 voltios.

```
import analogio

sensor_gas = analogio.AnalogIn(board.IO32)   # "escuchar" el pin IO32

print(sensor_gas.value)                      # ej.: 12000 en aire limpio
```

Por eso el gas no se mide en ppm: lo que llega es un número crudo, y ustedes deciden con la calibración desde qué valor hay peligro.

pwmio: hacer sonar el buzzer

El ESP32 no puede generar una onda de sonido "real", pero puede **prender y apagar un pin muy rápido**. Eso se llama **PWM** (modulación por ancho de pulso). Si el pin se prende y apaga 440 veces por segundo, el buzzer vibra a 440 Hz y suena la nota La.

```
import pwmio

buzzer = pwmio.PWMOut(board.IO26, frequency=440, variable_frequency=True)

buzzer.frequency = 2000     # tono agudo de alarma

buzzer.duty_cycle = 32768   # encendido la mitad del tiempo = suena

buzzer.duty_cycle = 0       # nunca encendido = silencio
```

- **`frequency`** define qué tan agudo suena.
- **`duty_cycle`** define si suena o no. La mitad de 65535 (32768) es la señal más limpia para un buzzer.

Así se tocan tanto la alarma (2000 Hz intermitente) como la canción de cuna, que es una lista de notas.

"Incluidas en CircuitPython" significa que estas dos librerías ya vienen en la placa y no hay que instalar nada.

adafruit_minimqtt: enviar datos a la app

Esta librería sí **hay que instalarla**: se copia en la carpeta `lib/` de la IdeaBoard. Permite que el peluche use **MQTT**, que funciona como un grupo de chat:

- El peluche **publica** mensajes en un "canal" (tópico), por ejemplo `nanyboard/grupo1/evento`.
- La app está **suscrita** a ese canal y recibe todo lo que se publica ahí.
- En medio hay un servidor llamado **broker** (`broker.hivemq.com`) que reparte los mensajes. El peluche y la app nunca se hablan directo.

```
mqtt.publish("nanyboard/grupo1/evento", '{"tipo": "ALERTA_CRITICA_GAS", "fase": "inicio"}')
```

Estructura del programa

El código se separó en archivos con responsabilidades claras, como se resume en la Tabla 1.

Tabla 1

Archivos del firmware de NanyBoard

Archivo	Propósito
code.py	Programa principal: inicializa el hardware, lee sensores cada 200 ms, evalúa alertas, controla salidas y publica por MQTT.
caras.py	Íconos de 5x5 (carita feliz, triste, peligro, clima, caída) y función para dibujarlos en la matriz.
settings.toml	Configuración: red Wi-Fi, servidor MQTT e identificador del peluche. Separa los datos sensibles del código.
pruebas/ prueba_sensores.py	Script de la sala de experimentos que imprime todas las lecturas de entrada y registra mínimos y máximos para calibrar.
pruebas/prueba_salidas.py	Script que recorre cada salida (matriz, LED RGB, buzzer, botón A) para verificarlas una por una.

Nota. El código completo y comentado se incluye en el Anexo A y los scripts de prueba en el Anexo B.

Lógica principal: máquina de estados con prioridades

El programa funciona como una máquina de estados. En cada ciclo se leen todos los sensores y la función evaluar() decide el estado del sistema siguiendo un orden de prioridad: si hay varias situaciones simultáneas, se atiende la más peligrosa. La Tabla 2 muestra los estados, la condición que los activa y la respuesta del peluche.

Tabla 2

Estados del sistema, condiciones y salidas asociadas

Orden	Estado	Condición	LED RGB	Matriz	Buzzer
1	ALERTA_CRITICA_GAS	Lectura del MQ-2 > UMBRAL_GAS	Rojo	Peligro (!)	Alarma intermitente
2	ALERTA_PANICO	Botón B presionado (10 s)	Rojo	Peligro (!)	—
3	ALERTA_CAIDA	Aceleración < 3 m/s² (caída libre) o > 25 m/s² (golpe); se mantiene 5 s	Amarillo	Flecha abajo	—
4	ALERTA_LLANTO	Ruido > UMBRAL_RUIDO durante 2 s seguidos	Amarillo	Carita triste	Canción de cuna
5	ADVERTENCIA_CLIMA	Temperatura > 28 °C o < 18 °C	Amarillo	Termómetro	—
—	NORMAL	Ninguna de las anteriores	Verde	Carita feliz	Apagado

Nota. El botón A silencia la alarma local (LED azul) hasta que el sistema vuelve al estado normal.

Cada vez que el estado cambia, el programa publica un evento de inicio y, al terminar, un evento de fin con la duración en segundos. Así la aplicación construye el historial pedido en los requerimientos del Avance 1 ("registro de cada evento y por cuánto tiempo estuvo activo"). Además, cada 5 segundos se envía la telemetría completa.

Fragmento clave comentado

La siguiente función es el núcleo de la toma de decisiones. Muestra el uso de condicionales anidados, variables de tiempo y el manejo de la prioridad entre alertas:

```
def evaluar(d):
    """Devuelve el estado según prioridad: gas > pánico > caída > llanto > clima."""
    global ruido_desde, caida_hasta
    ahora = time.monotonic()

    if d["gas"] > UMBRAL_GAS:                # 1. lo más peligroso primero
        return GAS
    if ahora < panico_hasta:                 # 2. botón B activo
        return PANICO
    if d["movimiento"] < CAIDA_LIBRE or d["movimiento"] > GOLPE_FUERTE:
        caida_hasta = ahora + 5              # 3. la caída se "recuerda" 5 s
    if ahora < caida_hasta:
        return CAIDA

    # 4. Llanto = ruido alto SOSTENIDO (evita alertas por un portazo)
    if d["ruido"] > UMBRAL_RUIDO:
        ruido_desde = ruido_desde or ahora
        if ahora - ruido_desde >= SEG_RUIDO_SOSTENIDO:
            return LLANTO
    else:
        ruido_desde = None

    if d["temp"] > TEMP_MAX or d["temp"] < TEMP_MIN:   # 5. confort térmico
        return CLIMA
    return NORMAL
```

Cambios respecto al pseudocódigo del Avance 1

Al pasar del pseudocódigo al código real se hicieron los ajustes de la Tabla 3. La mayoría surgieron del análisis de limitaciones del hardware y de la revisión del código con apoyo de IA (ver sección de bitácora).

Tabla 3

Ajustes entre el diseño del Avance 1 y la implementación

Avance 1	Avance 2	Justificación
Umbrales en ppm (gas) y dB (ruido)	Umbrales en valores crudos de 0 a 65535, calibrados en sala	El MQ-2 y un micrófono analógico no entregan ppm ni dB directamente; convertirlos exige curvas de calibración de laboratorio.
Alerta de llanto con una sola lectura alta	Ruido sostenido por 2 s	Un portazo o un juguete que cae generaba falsas alarmas.
Notificación en cada ciclo de 200 ms	Notificación solo al cambiar de estado	Evita enviar cinco mensajes por segundo y saturar la aplicación.
Acelerómetro sin regla definida	Detección de caída libre y de golpe fuerte	Se aprovecha el sensor de la IdeaSense que ya estaba en el diseño.
Botón B como pánico/reinicio	Botón B activa alerta de pánico por 10 s	Comportamiento definido y verificable en pruebas.
Melodía y alarma con pausas (sleep)	Reproductor de sonido no bloqueante	Con pausas el programa dejaba de leer sensores mientras sonaba la música.
Requiere Wi-Fi	Modo local si no hay red, con reconexión cada 15 s	Si el Wi-Fi falla, el peluche sigue alertando con luz, matriz y sonido.

Montaje preliminar

Componentes utilizados

Tabla 4

Componentes del prototipo preliminar

Componente	Función	Conexión
IdeaBoard (ESP32)	Controlador principal, Wi-Fi, LED RGB	—
IdeaSense Explorer	Matriz 5x5, botones, temperatura/humedad, luz, acelerómetro	Cable STEMMA QT / Qwiic (I2C)
Sensor MQ-2	Gas y humo	Analógico
Micrófono analógico	Nivel de ruido / llanto	Analógico
Buzzer pasivo	Alarma y canción de cuna	PWM
Batería recargable	Alimentación portátil	Entrada DC de la IdeaBoard

Tabla de conexiones (pinout preliminar)

Tabla 5

Conexiones de los sensores externos a la IdeaBoard

Componente	Pin del componente	Pin de la IdeaBoard	Observación
IdeaSense Explorer	Conector Qwiic	Conector STEMMA QT	I2C: direcciones 0x70, 0x44, 0x29, 0x6A
MQ-2	VCC	5 V	El calentador interno necesita 5 V
MQ-2	GND	GND	—
MQ-2	AO	IO32 (a través de divisor de voltaje)	Divisor 10 kΩ / 20 kΩ para bajar de 5 V a 3,3 V
Micrófono	VCC / GND	3,3 V / GND	—
Micrófono	OUT	IO35	Pin solo de entrada, ADC1
Buzzer	+ / −	IO26 / GND	Salida PWM

Nota. Los pines analógicos se eligieron en el grupo ADC1 (GPIO 32 a 39) porque el ESP32 no puede usar el ADC2 mientras el Wi-Fi está encendido. Los números de pin son preliminares y se verificarán en la placa con print(dir(board)).

Etapas del montaje

El montaje se planificó en tres etapas para no arriesgar el hardware dentro del peluche antes de validar el funcionamiento:

- **Etapa 1, en mesa (este avance):** IdeaBoard y IdeaSense conectadas por cable STEMMA QT, y sensores externos en protoboard. Permite probar cada componente por separado.
- **Etapa 2, chasis interno (este avance, preliminar):** caja de cartón reciclado que protege las placas de la presión al abrazar el peluche, con aberturas para la matriz LED, el sensor de luz y la entrada de aire del MQ-2.
- **Etapa 3, integración en el peluche (entrega final):** chasis dentro del peluche, apertura trasera con velcro para acceso a la batería y malla en el pecho para la matriz.

Sala de experimentos: pruebas de sensores y salidas

Procedimiento

Las pruebas se realizan en tres niveles. Primero, cada entrada se verifica con el script prueba_sensores.py, observando las lecturas en la consola serial. Segundo, cada salida se verifica con prueba_salidas.py. Tercero, con el programa completo (code.py) se ejecutan escenarios que combinan entradas y salidas. En cada prueba se anota el resultado obtenido y si cumple el resultado esperado.

Tabla 6

Pruebas de entradas (sensores y botones)

ID	Sensor	Procedimiento	Resultado esperado	Resultado obtenido	¿Cumple?
E1	Temperatura	Lectura en reposo durante 1 min	Valor estable similar a la temperatura del aula		
E2	Temperatura	Acercar la mano o un secador por 30 s	La lectura sube varios grados		
E3	Humedad	Soplar cerca del sensor	La humedad sube		
E4	Luz	Tapar el sensor con la mano	El valor baja notablemente		
E5	Acelerómetro	Placa quieta sobre la mesa	Magnitud ≈ 9,8 m/s²		
E6	Acelerómetro	Dejar caer el peluche 30 cm sobre un cojín	Magnitud < 3 o > 25 m/s²		
E7	Botón A	Presionar el botón	Consola: "Botón A presionado"		
E8	Botón B	Presionar el botón	Consola: "Botón B presionado"		
E9	MQ-2	Aire limpio, tras 3 min de calentamiento	Valor base estable (anotarlo)		
E10	MQ-2	Gas de un encendedor SIN llama a 10 cm	El valor sube claramente sobre la base		
E11	Micrófono	Habitación en silencio	Amplitud baja (anotarla)		
E12	Micrófono	Reproducir una grabación de llanto cerca	Amplitud alta (anotarla)		

Nota. Por seguridad, la prueba E10 se hace con el gas del encendedor sin encenderlo, en un lugar ventilado y nunca con llama cerca del peluche.

Tabla 7

Pruebas de salidas

ID	Salida	Procedimiento	Resultado esperado	Resultado obtenido	¿Cumple?
S1	Matriz 5x5	Mostrar FELIZ	Carita feliz derecha y legible		
S2	Matriz 5x5	Mostrar TRISTE	Carita triste derecha y legible		
S3	Matriz 5x5	Mostrar PELIGRO, CLIMA y CAIDA	Los tres íconos se distinguen		
S4	LED RGB	Ciclo de colores	Verde, amarillo, rojo y azul		
S5	Buzzer	Tono de alarma	Pitido agudo intermitente		
S6	Buzzer	Canción de cuna	Melodía reconocible y suave		
S7	Botón A	Presionar durante la ventana de 5 s	Consola: "Botón A detectado: OK"		

Tabla 8

Pruebas de integración con el programa completo (code.py)

ID	Escenario	Resultado esperado	Resultado obtenido	¿Cumple?
I1	Encender en condiciones normales	LED verde, carita feliz, buzzer apagado		
I2	Gas del encendedor (sin llama)	LED rojo, ícono de peligro, alarma, evento enviado		
I3	Botón A durante la alarma de gas	Alarma silenciada, LED azul		
I4	Grabación de llanto por más de 2 s	LED amarillo, carita triste, canción de cuna		
I5	Aplauso o portazo aislado	NO se activa la alerta de llanto		
I6	Soltar el peluche sobre un cojín	Alerta de caída por 5 s y regreso a normal		
I7	Calentar el sensor sobre 28 °C	Advertencia de clima (LED amarillo, termómetro)		
I8	Presionar el botón B	Alerta de pánico por 10 s		
I9	Apagar el router Wi-Fi	El peluche sigue alertando de forma local		
I10	Telemetría por MQTT	Las lecturas llegan cada 5 s al cliente MQTT o a la app		

Calibración de umbrales

Con los valores registrados en las pruebas E9 a E12 se fijan los umbrales definitivos en code.py. Como regla práctica, el umbral se coloca aproximadamente a la mitad entre el valor base y el valor medido durante el evento.

Tabla 9
Registro de calibración

Variable	Valor base (normal)	Valor durante el evento	Umbral elegido
UMBRAL_GAS			
UMBRAL_RUIDO			
CAIDA_LIBRE / GOLPE_FUERTE	≈ 9,8 m/s²		

Problemas encontrados y soluciones

Tabla 10
Registro de incidencias durante las pruebas

Problema observado	Causa identificada	Solución aplicada

Bitácora DiálogoX™ / IA: consultas técnicas, depuración y optimización

En esta fase se utilizó Claude (Anthropic) como asistente de programación. Se le compartió el documento del Avance 1 y se le pidió ayuda para pasar del pseudocódigo a código real. La Tabla 11 registra las consultas, el aporte de la IA y el análisis crítico del equipo, es decir, qué se aceptó, qué se verificó y qué se cambió.

Tabla 11

Registro de interacción con IA

#	Consulta del equipo	Aporte de la IA	Análisis crítico y decisión del equipo
1	¿Cómo empezar el código conectado a la IdeaSense, con los periféricos y la app?	Propuso una arquitectura: firmware en CircuitPython, comunicación MQTT y app en Flutter, con un simulador para avanzar sin hardware.	Elegimos CircuitPython por ser el lenguaje oficial de la IdeaBoard. La app es un complemento; el foco del curso es la placa.
2	¿Cómo se leen los sensores de la IdeaSense?	Consultó el repositorio oficial de CENFOTEC y usó la API real: idea.temp, idea.humid, idea.light, idea.accel, idea.matrix e idea.events.	Verificamos que la fuente fuera oficial (repositorio Universidad-Cenfotec/ IdeaSense) y no código inventado.
3	Depuración preventiva: ¿en qué pines conectar los sensores analógicos?	Advirtió que con Wi-Fi activo el ESP32 solo lee analógico en ADC1 (GPIO 32 a 39).	Sin esta advertencia el MQ-2 habría dado lecturas erróneas sin una causa evidente. Movimos los analógicos a IO32 e IO35.
4	Riesgo eléctrico del MQ-2	Señaló que su salida puede llegar a 5 V y que el ESP32 solo tolera 3,3 V; recomendó un divisor de voltaje.	Agregamos el divisor al montaje para proteger la placa.
5	¿Se puede medir en ppm y dB como en el Avance 1?	Explicó que los sensores entregan valores crudos y propuso calibrar en la sala de experimentos.	Corregimos el pseudocódigo y diseñamos la Tabla 9 de calibración.
6	Optimización: alertas repetidas	Detectó que el pseudocódigo enviaba una notificación cada 200 ms y propuso enviarla solo al cambiar de estado, con eventos de inicio y fin.	Esto además resolvió el requerimiento de registrar la duración de cada evento.
7	Optimización: falsas alarmas	Propuso exigir ruido sostenido por 2 s y mantener la alerta de caída por 5 s.	Lo validaremos con las pruebas I5 e I6.
8	Optimización: la melodía bloqueaba el programa	Reemplazó las pausas por un reproductor no bloqueante que avanza una nota por ciclo.	Entendimos la diferencia entre código bloqueante y no bloqueante.
9	¿Qué pasa si se cae el Wi-Fi?	Agregó un modo local, reconexión cada 15 s y mensaje offline automático (last will).	Es clave para la demostración final, que exige operar sin fallos críticos.
10	¿Cómo saber si el código funciona antes de tener la placa?	Ejecutó el programa en una computadora simulando el hardware y comprobó la transición NORMAL → CLIMA → NORMAL y los mensajes enviados.	Es una verificación de lógica, no de hardware. Las pruebas reales siguen siendo necesarias.

Limitaciones detectadas en la ayuda de la IA

- La IA no conocía el diagrama exacto de pines de la IdeaBoard; los pines propuestos son supuestos que el equipo debe verificar en la placa.
- No pudo garantizar la orientación de la matriz LED; puede ser necesario invertir las coordenadas.
- Los umbrales iniciales son estimaciones; solo la calibración en sala los vuelve confiables.

Próximos pasos hacia la entrega final

- Completar las pruebas y la calibración, y actualizar los umbrales en code.py.
- Integrar el chasis dentro del peluche (etapa 3) y ordenar el cableado.
- Elaborar el diagrama de conexiones definitivo y la galería fotográfica.
- Conectar la aplicación móvil para mostrar indicadores, historial y el botón de canción de cuna.
- Evaluar la integración de la cámara ESP32-CAM como placa independiente.
- Incorporar la retroalimentación de la docente sobre el Avance 1.

Referencias

Adafruit Industries. (s. f.). Adafruit MiniMQTT library [Documentación]. https://docs.circuitpython.org/projects/minimqtt/en/latest/

Anthropic. (2026). Claude [Modelo de lenguaje de gran tamaño]. https://claude.ai

CircuitPython. (s. f.). IdeaBoard by CRCibernetica. https://circuitpython.org/board/crcibernetica_ideaboard/

CRCibernética. (s. f.). IdeaCode: CircuitPython IDE. https://ideacode.crcibernetica.com/

Espressif Systems. (s. f.). ESP-IDF programming guide: Analog to digital converter (ADC). https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc_oneshot.html

Universidad CENFOTEC. (2026). IdeaSense [Repositorio de código fuente]. GitHub. https://github.com/Universidad-Cenfotec/IdeaSense

Anexo A. Código fuente del firmware

A.1 code.py

```
# ============================================================
#  NanyBoard - Firmware de la IdeaBoard + IdeaSense Explorer
#  CircuitPython. Copiar a la raíz de la unidad CIRCUITPY.
#
#  Flujo: leer sensores -> evaluar alertas -> salidas locales
#         (matriz, LED RGB, buzzer) -> publicar por MQTT a la app.
# ============================================================
import time
import json
import os
import math

import board
import analogio
import pwmio
import wifi
import socketpool
import adafruit_minimqtt.adafruit_minimqtt as MQTT

from ideaboard import IdeaBoard
from ideasense import IdeaSense
import caras

# ------------------------------------------------------------
# 1. CONFIGURACIÓN
# ------------------------------------------------------------
DEVICE_ID = os.getenv("DEVICE_ID", "nanyboard-demo")
BROKER = os.getenv("MQTT_BROKER", "broker.hivemq.com")
PUERTO = int(os.getenv("MQTT_PORT", 1883))

# Tópicos MQTT (la app usa exactamente los mismos)
T_TELEMETRIA = "nanyboard/%s/telemetria" % DEVICE_ID   # cada 5 s
T_EVENTO = "nanyboard/%s/evento" % DEVICE_ID           # inicio/fin de alertas
T_COMANDO = "nanyboard/%s/comando" % DEVICE_ID         # app -> peluche
T_ESTADO = "nanyboard/%s/estado" % DEVICE_ID           # online/offline (retenido)

# Umbrales. OJO: el MQ-2 y el micrófono dan valores CRUDOS (0-65535),
# no ppm ni dB. Hay que calibrarlos midiendo en la habitación real.
UMBRAL_GAS = 30000        # crudo; calibrar con aire limpio vs. encendedor sin llama
UMBRAL_RUIDO = 12000      # amplitud pico a pico; calibrar con llanto grabado
SEG_RUIDO_SOSTENIDO = 2   # segundos de ruido continuo antes de alertar "llanto"
TEMP_MAX = 28.0
TEMP_MIN = 18.0
CAIDA_LIBRE = 3.0         # m/s^2: magnitud baja = el peluche va cayendo
GOLPE_FUERTE = 25.0       # m/s^2: magnitud alta = impacto o sacudida

PERIODO_LOOP = 0.2        # 200 ms, como en el pseudocódigo
PERIODO_TELEMETRIA = 5.0  # 5 s

# Pines de los sensores EXTERNOS. Verificar nombres con:
#   import board; print(dir(board))
# Con Wi-Fi activo, el ESP32 solo puede leer analógico en ADC1 (GPIO 32-39).
# Si un pin no existe o el sensor no está conectado, se deja en None y se omite.
PIN_GAS = getattr(board, "IO32", None)
PIN_MIC = getattr(board, "IO35", None)
PIN_BUZZER = getattr(board, "IO26", None)

# ------------------------------------------------------------
# 2. INICIALIZACIÓN DE HARDWARE
# ------------------------------------------------------------
ib = IdeaBoard()     # LED RGB y pines de la IdeaBoard
idea = IdeaSense()   # matriz 5x5, botones, temp/humedad, luz, acelerómetro
idea.matrix.brightness(0.3)

sensor_gas = analogio.AnalogIn(PIN_GAS) if PIN_GAS else None
sensor_mic = analogio.AnalogIn(PIN_MIC) if PIN_MIC else None
buzzer = (pwmio.PWMOut(PIN_BUZZER, frequency=440, duty_cycle=0,
```

```
                        variable_frequency=True) if PIN_BUZZER else None)

VERDE, AMARILLO, ROJO, AZUL = (0, 255, 0), (255, 160, 0), (255, 0, 0), (0, 0, 255)


# ------------------------------------------------------------
# 3. LECTURA DE SENSORES
# ------------------------------------------------------------
def leer_gas():
    return sensor_gas.value if sensor_gas else 0


def leer_ruido(muestras=60):
    """Amplitud pico a pico del micrófono: más alto = más ruido."""
    if not sensor_mic:
        return 0
    minimo, maximo = 65535, 0
    for _ in range(muestras):
        v = sensor_mic.value
        minimo = min(minimo, v)
        maximo = max(maximo, v)
    return maximo - minimo


def leer_movimiento():
    """Devuelve la magnitud de la aceleración en m/s^2 (~9.8 en reposo)."""
    ax, ay, az = idea.accel
    return math.sqrt(ax * ax + ay * ay + az * az)


def leer_todo():
    return {
        "gas": leer_gas(),
        "temp": round(idea.temp, 1),
        "humedad": round(idea.humid, 1),
        "luz": idea.light,
        "movimiento": round(leer_movimiento(), 2),
        "ruido": leer_ruido(),
    }


# ------------------------------------------------------------
# 4. BUZZER NO BLOQUEANTE (alarma y canción de cuna)
# ------------------------------------------------------------
# Notas (frecuencia Hz, duración s). 0 = silencio.
CANCION_CUNA = [(392, .4), (392, .4), (440, .8), (392, .8), (523, .8), (494, 1.2),
                (0, .3), (392, .4), (392, .4), (440, .8), (392, .8), (587, .8), (523, 1.2)]
ALARMA = [(2000, .25), (0, .15)]


class Sonido:
    def __init__(self):
        self.notas = []
        self.i = 0
        self.fin_nota = 0
        self.repetir = False

    def tocar(self, notas, repetir=False):
        self.notas, self.i, self.repetir, self.fin_nota = notas, 0, repetir, 0

    def detener(self):
        self.notas = []
        if buzzer:
            buzzer.duty_cycle = 0

    def actualizar(self):
        if not buzzer or not self.notas or time.monotonic() < self.fin_nota:
            return
        if self.i >= len(self.notas):
            if not self.repetir:
                self.detener()
                return
```

```
            self.i = 0
        freq, dur = self.notas[self.i]
        if freq:
            buzzer.frequency = freq
            buzzer.duty_cycle = 32768  # 50 %
        else:
            buzzer.duty_cycle = 0
        self.fin_nota = time.monotonic() + dur
        self.i += 1

    @property
    def sonando(self):
        return bool(self.notas)


sonido = Sonido()

# ---------------------------------------------------------------
# 5. MQTT (Wi-Fi + comunicación con la app)
# ---------------------------------------------------------------
mqtt = None


def al_recibir(cliente, topico, mensaje):
    """Comandos que manda la app: 'cuna', 'silenciar', 'detener_cuna', 'ping'."""
    print("Comando:", mensaje)
    if mensaje == "cuna":
        sonido.tocar(CANCION_CUNA)
    elif mensaje in ("silenciar", "detener_cuna"):
        silenciar()
    elif mensaje == "ping":
        publicar(T_TELEMETRIA, dict(leer_todo(), estado=estado_actual))


def conectar():
    global mqtt
    if not wifi.radio.connected:
        print("Conectando a Wi-Fi...")
        wifi.radio.connect(os.getenv("CIRCUITPY_WIFI_SSID"),
                           os.getenv("CIRCUITPY_WIFI_PASSWORD"))
        print("IP:", wifi.radio.ipv4_address)
    pool = socketpool.SocketPool(wifi.radio)
    mqtt = MQTT.MQTT(broker=BROKER, port=PUERTO, socket_pool=pool,
                     client_id=DEVICE_ID, keep_alive=30, socket_timeout=0.05)
    # Si el peluche se apaga o pierde conexión, el broker avisa a la app.
    mqtt.will_set(T_ESTADO, "offline", retain=True)
    mqtt.on_message = al_recibir
    mqtt.connect()
    mqtt.subscribe(T_COMANDO)
    mqtt.publish(T_ESTADO, "online", retain=True)
    print("MQTT conectado a", BROKER)


def publicar(topico, datos):
    try:
        if mqtt:
            mqtt.publish(topico, json.dumps(datos))
    except Exception as e:
        print("Error publicando:", e)


# ---------------------------------------------------------------
# 6. LÓGICA DE ALERTAS
# ---------------------------------------------------------------
NORMAL = "NORMAL"
GAS = "ALERTA_CRITICA_GAS"
CAIDA = "ALERTA_CAIDA"
LLANTO = "ALERTA_LLANTO"
CLIMA = "ADVERTENCIA_CLIMA"
PANICO = "ALERTA_PANICO"

estado_actual = NORMAL
```

```
inicio_estado = time.monotonic()
silenciado = False
ruido_desde = None
panico_hasta = 0
caida_hasta = 0   # una caída se mantiene 5 s para no "parpadear" entre estados


def evaluar(d):
    """Devuelve el estado según prioridad: gas > pánico > caída > llanto > clima."""
    global ruido_desde, caida_hasta
    ahora = time.monotonic()

    if d["gas"] > UMBRAL_GAS:
        return GAS
    if ahora < panico_hasta:
        return PANICO
    if d["movimiento"] < CAIDA_LIBRE or d["movimiento"] > GOLPE_FUERTE:
        caida_hasta = ahora + 5
    if ahora < caida_hasta:
        return CAIDA

    # Llanto = ruido alto SOSTENIDO (evita alertas por un golpe de puerta)
    if d["ruido"] > UMBRAL_RUIDO:
        ruido_desde = ruido_desde or ahora
        if ahora - ruido_desde >= SEG_RUIDO_SOSTENIDO:
            return LLANTO
    else:
        ruido_desde = None

    if d["temp"] > TEMP_MAX or d["temp"] < TEMP_MIN:
        return CLIMA
    return NORMAL


def aplicar_salidas(estado):
    """Matriz, LED RGB y buzzer según el estado (solo al cambiar de estado)."""
    if estado == GAS:
        ib.pixel = ROJO
        caras.mostrar(idea.matrix, caras.PELIGRO)
        if not silenciado:
            sonido.tocar(ALARMA, repetir=True)
    elif estado == PANICO:
        ib.pixel = ROJO
        caras.mostrar(idea.matrix, caras.PELIGRO)
    elif estado == CAIDA:
        ib.pixel = AMARILLO
        caras.mostrar(idea.matrix, caras.CAIDA)
    elif estado == LLANTO:
        ib.pixel = AMARILLO
        caras.mostrar(idea.matrix, caras.TRISTE)
        sonido.tocar(CANCION_CUNA)  # intenta calmar al bebé
    elif estado == CLIMA:
        ib.pixel = AMARILLO
        caras.mostrar(idea.matrix, caras.CLIMA)
    else:
        ib.pixel = VERDE
        caras.mostrar(idea.matrix, caras.FELIZ)
        if sonido.notas is ALARMA:
            sonido.detener()


def silenciar():
    global silenciado
    silenciado = True
    sonido.detener()
    ib.pixel = AZUL


def cambiar_estado(nuevo, d):
    """Registra fin del evento anterior e inicio del nuevo (para el historial)."""
    global estado_actual, inicio_estado, silenciado
    ahora = time.monotonic()
```

```
    if estado_actual != NORMAL:
        publicar(T_EVENTO, {"tipo": estado_actual, "fase": "fin",
                            "duracion_s": round(ahora - inicio_estado)})
    if nuevo != NORMAL:
        publicar(T_EVENTO, {"tipo": nuevo, "fase": "inicio", "lecturas": d})
        # TODO: pedir foto a la ESP32-CAM (otra placa) cuando nuevo == GAS
    else:
        silenciado = False
    estado_actual, inicio_estado = nuevo, ahora
    aplicar_salidas(nuevo)
    print("Estado ->", nuevo)


def revisar_botones():
    global panico_hasta
    evento = idea.events.get()
    if evento and evento.pressed:
        if evento.key_number == 0:      # Botón A: silenciar alarma local
            silenciar()
        elif evento.key_number == 1:    # Botón B: pánico manual (10 s)
            panico_hasta = time.monotonic() + 10


# ------------------------------------------------------------
# 7. CICLO PRINCIPAL
# ------------------------------------------------------------
aplicar_salidas(NORMAL)
try:
    conectar()
except Exception as e:
    print("Sin conexión, sigo en modo local:", e)

ultima_telemetria = 0
ultimo_reintento = time.monotonic()

while True:
    inicio = time.monotonic()

    datos = leer_todo()
    revisar_botones()

    nuevo = evaluar(datos)
    if nuevo != estado_actual:
        cambiar_estado(nuevo, datos)

    if inicio - ultima_telemetria >= PERIODO_TELEMETRIA:
        publicar(T_TELEMETRIA, dict(datos, estado=estado_actual))
        ultima_telemetria = inicio

    # Mantener MQTT vivo y recibir comandos; reconectar si se cae
    try:
        if mqtt:
            mqtt.loop(timeout=0.05)
    except Exception as e:
        print("MQTT caído:", e)
        mqtt = None
    if mqtt is None and inicio - ultimo_reintento > 15:
        ultimo_reintento = inicio
        try:
            conectar()
        except Exception as e:
            print("Reintento fallido:", e)

    sonido.actualizar()

    espera = PERIODO_LOOP - (time.monotonic() - inicio)
    if espera > 0:
        time.sleep(espera)
```

A.2 caras.py

```
# Íconos 5x5 para la matriz LED de la IdeaSense.
# Cada string es una fila (de arriba hacia abajo); "1" = LED encendido.

FELIZ = (
    "01010",
    "01010",
    "00000",
    "10001",
    "01110",
)

TRISTE = (
    "01010",
    "01010",
    "00000",
    "01110",
    "10001",
)

PELIGRO = (  # signo de exclamación
    "00100",
    "00100",
    "00100",
    "00000",
    "00100",
)

CLIMA = (  # termómetro
    "00100",
    "00100",
    "00100",
    "01110",
    "01110",
)

CAIDA = (  # flecha hacia abajo
    "00100",
    "00100",
    "10101",
    "01110",
    "00100",
)


def mostrar(matriz, icono):
    """Dibuja un ícono en la matriz HT16K33 de la IdeaSense."""
    matriz.fill(0)
    for y, fila in enumerate(icono):
        for x, celda in enumerate(fila):
            if celda == "1":
                matriz[x, y] = 1
    matriz.show()
```

A.3 settings.toml (sin credenciales reales)

```
# Configuración de NanyBoard (CircuitPython lee este archivo con os.getenv)
# Copiar a la raíz de la IdeaBoard (unidad CIRCUITPY) junto a code.py

CIRCUITPY_WIFI_SSID = "NOMBRE_DE_TU_WIFI"
CIRCUITPY_WIFI_PASSWORD = "CLAVE_DE_TU_WIFI"

# Broker MQTT público para el prototipo (no usar para datos reales)
MQTT_BROKER = "broker.hivemq.com"
MQTT_PORT = 1883

# Identificador único del peluche (cámbienlo para no chocar con otros grupos)
DEVICE_ID = "nanyboard-grupo1"
```

Anexo B. Scripts de la sala de experimentos

B.1 prueba_sensores.py

```
# ============================================================
#  NanyBoard - PRUEBA DE ENTRADAS (Avance 2, Sala de Experimentos)
#  Uso: copiar este contenido como code.py en la IdeaBoard
#  (o ejecutarlo desde IdeaCode) y mirar la consola serial.
#  Imprime cada 0,5 s todas las lecturas para llenar la tabla
#  de pruebas E1-E12 del documento.
# ============================================================
import time
import math
import board
import analogio
from ideasense import IdeaSense

idea = IdeaSense()

# Sensores externos (mismos pines que code.py). Si un pin no existe,
# se muestra "--" en vez de detener el programa.
def abrir_analogico(nombre):
    pin = getattr(board, nombre, None)
    return analogio.AnalogIn(pin) if pin else None

gas = abrir_analogico("IO32")
mic = abrir_analogico("IO35")

print("Pines disponibles en esta placa:")
print([p for p in dir(board) if not p.startswith("_")])
print()

# Para calibrar: se guardan el mínimo y el máximo vistos de gas y ruido
gas_min, gas_max = 65535, 0
ruido_max = 0


def amplitud_mic(muestras=60):
    if not mic:
        return None
    lo, hi = 65535, 0
    for _ in range(muestras):
        v = mic.value
        lo, hi = min(lo, v), max(hi, v)
    return hi - lo


while True:
    ax, ay, az = idea.accel
    magnitud = math.sqrt(ax * ax + ay * ay + az * az)

    g = gas.value if gas else None
    r = amplitud_mic()
    if g is not None:
        gas_min, gas_max = min(gas_min, g), max(gas_max, g)
    if r is not None:
        ruido_max = max(ruido_max, r)

    print("Temp: %.1f C | Hum: %.1f %% | Luz: %d | Accel: %.2f m/s2 | Gas: %s (min %s / max %s) |
Ruido: %s (max %s)" % (
        idea.temp, idea.humid, idea.light, magnitud,
        g if g is not None else "--", gas_min if gas else "--", gas_max if gas else "--",
        r if r is not None else "--", ruido_max if mic else "--"))

    evento = idea.events.get()
    if evento and evento.pressed:
        print(">>> Botón", "ABC"[evento.key_number], "presionado")

    time.sleep(0.5)
```

B.2 prueba_salidas.py

```
# ============================================================
#  NanyBoard - PRUEBA DE SALIDAS (Avance 2, Sala de Experimentos)
#  Uso: copiar como code.py junto con caras.py.
#  Recorre cada salida y anuncia en la consola qué debería verse
#  u oírse, para llenar la tabla de pruebas S1-S7 del documento.
# ============================================================
import time
import board
import pwmio
from ideaboard import IdeaBoard
from ideasense import IdeaSense
import caras

ib = IdeaBoard()
idea = IdeaSense()
idea.matrix.brightness(0.3)

pin_buzzer = getattr(board, "IO26", None)
buzzer = (pwmio.PWMOut(pin_buzzer, frequency=440, duty_cycle=0,
                       variable_frequency=True) if pin_buzzer else None)


def paso(nombre):
    print("\n==>", nombre)
    time.sleep(0.3)


def tono(freq, dur):
    if not buzzer:
        print("   (sin buzzer conectado)")
        time.sleep(dur)
        return
    buzzer.frequency = freq
    buzzer.duty_cycle = 32768
    time.sleep(dur)
    buzzer.duty_cycle = 0


while True:
    paso("S1 Matriz: carita FELIZ")
    caras.mostrar(idea.matrix, caras.FELIZ)
    time.sleep(2)

    paso("S2 Matriz: carita TRISTE")
    caras.mostrar(idea.matrix, caras.TRISTE)
    time.sleep(2)

    paso("S3 Matriz: íconos PELIGRO, CLIMA y CAIDA")
    for icono in (caras.PELIGRO, caras.CLIMA, caras.CAIDA):
        caras.mostrar(idea.matrix, icono)
        time.sleep(1.5)

    paso("S4 LED RGB: verde, amarillo, rojo, azul")
    for nombre, color in (("verde", (0, 255, 0)), ("amarillo", (255, 160, 0)),
                          ("rojo", (255, 0, 0)), ("azul", (0, 0, 255))):
        print("   ", nombre)
        ib.pixel = color
        time.sleep(1)
    ib.pixel = (0, 0, 0)

    paso("S5 Buzzer: tono de alarma (2000 Hz intermitente)")
    for _ in range(4):
        tono(2000, 0.25)
        time.sleep(0.15)

    paso("S6 Buzzer: canción de cuna")
    for f, d in [(392, .4), (392, .4), (440, .8), (392, .8), (523, .8), (494, 1.2)]:
        tono(f, d)

    paso("S7 Presione el botón A en los próximos 5 s (silenciar)")
```

```
    limite = time.monotonic() + 5
    while time.monotonic() < limite:
        ev = idea.events.get()
        if ev and ev.pressed and ev.key_number == 0:
            print("   Botón A detectado: OK")
            break
    else:
        print("   No se detectó el botón A")

    idea.matrix.fill(0)
    idea.matrix.show()
    print("\nCiclo de pruebas completo. Se repite en 3 s...")
    time.sleep(3)
```